

PM1-AA3-EV05 — Informe de depuración

Plan de Mejoramiento · Depuración y complemento de la app móvil de e-commerce

APRENDIZ Daniel Salas Roman	FICHA 3223873	RETOS CORREGIDOS Reto 1 y Reto 2	ARCHIVOS MODIFICADOS 9
--------------------------------	------------------	-------------------------------------	---------------------------

1. Hipótesis contra hallazgo

En la evidencia PM1-AA1-EV01 planteé cinco hipótesis antes de tocar el código. Ésta es la comparación con lo que realmente encontré, y cómo comprobé cada una.

HIPÓTESIS INICIAL	RESULTADO	CÓMO LO COMPROBÉ
El endpoint no existe, cambió de ruta o devuelve 404 / 500.	Descartada	<code>curl -i http://127.0.0.1:5050/api/locations/countries</code> respondió 200 OK con los 5 países completos. El servidor funcionaba perfectamente.
La petición nunca se dispara porque nadie llama a la función de carga.	Descartada	<code>LocationManagementViewModel</code> tiene <code>init { loadCountries() }</code> , así que la carga se dispara al construirse el <code>ViewModel</code> . Además, la terminal del backend registró la petición <code>GET /api/locations/countries 200</code> : la petición sí salía y sí llegaba.
Los nombres de la <code>data class</code> no coinciden con los del JSON.	Descartada (pero cerca)	Los <code>@SerializedName</code> de <code>Country</code> eran correctos: <code>id_Country</code> y <code>CountryName</code> coinciden exactamente con el JSON. El desajuste no estaba en los campos , sino un nivel más arriba: en la forma de la respuesta. Esta hipótesis apuntaba en la dirección correcta pero al lugar equivocado.
El estado del <code>ViewModel</code> se actualiza pero la pantalla no lo observa.	Descartada	<code>LocationManagementScreen</code> lee <code>viewModel.countries</code> , que es exactamente la misma propiedad <code>mutableStateOf</code> que escribe <code>loadCountries()</code> . No había dos variables distintas.
Falta el token y el servidor responde 401.	Descartada	Las rutas de ubicaciones son públicas: <code>curl</code> sin cabecera <code>Authorization</code> devolvió 200. El 401 sólo aparece en las rutas de perfil y usuarios.
Hallazgo real El tipo declarado en Retrofit no coincide con la forma del JSON que devuelve el servidor.	Confirmada	Comparando lado a lado la respuesta de <code>curl</code> (un objeto <code>{...}</code>) contra la firma <code>Response<List<Country>></code> (un arreglo <code>[...]</code>). Gson lanzaba <code>JsonSyntaxException: Expected BEGIN_ARRAY but was BEGIN_OBJECT</code> .

Lo que más me costó ver

Ninguna de mis cinco hipótesis mencionaba la *forma* de la respuesta: todas hablaban de la ruta, del disparo, de los nombres de los campos, del estado o del token. Es decir, yo estaba buscando el error en los extremos de la cadena, y estaba justo en la mitad, en la traducción de JSON a objeto Kotlin. La pista que lo destapó fue comparar el JSON crudo de `curl` con la firma del método, algo que no se puede hacer mirando sólo el código de Android.

2. Causa raíz

Reto 1 — Desplegables de países y estados

El backend Flask responde siempre con la colección envuelta: `jsonify({'countries': [...], 'count': n})`. Sin embargo, `ApiService.kt` declaraba `suspend fun getCountries(): Response<List<Country>>`, es decir, le prometía a Retrofit que el cuerpo sería un arreglo. Al recibir un objeto, Gson lanzaba `JsonSyntaxException`; esa excepción caía en el `catch` del `ViewModel`, que la convertía en el mensaje «*Error de conexión*» —un mensaje engañoso, porque la conexión había funcionado sin problema— y la lista de estado se quedaba vacía. Como el `ExposedDropDownMenu` sólo dibuja lo que hay en esa lista, el desplegable aparecía sin elementos. Lo mismo ocurría con estados y ciudades.

La prueba de que fue un descuido y no un cambio de la API: en ese mismo archivo, las colecciones de usuarios, categorías y productos ya usaban clases envoltorio (`UsersResponse`, `CategoriesResponse`, `ProductsResponse`). Sólo ubicaciones y roles eran la excepción.

Reto 2 — Cambio de rol y edición de perfil

Exactamente el mismo error, repetido en cuatro endpoints más:

- `getRoles()` devolvía `Response<List<Role>>`, pero el backend responde `{"roles":[...]}`. Sin roles cargados, el desplegable de la pantalla de gestión de usuarios quedaba vacío y era imposible asignar ningún rol.
- `getProfile()` devolvía `Response<UserProfile>`, pero el backend responde `{"user":{...}}`. Gson buscaba `id_User`, `UserName` y `Email` en el nivel de afuera; están una capa más adentro. La pantalla de perfil se abría en blanco.
- `updateProfile()` y `updateUserRoles()` devolvían el mismo tipo suelto, cuando el backend responde `{"message":"...", "user":{...}}`.

Además de los envoltorios, la edición de perfil tenía dos carencias propias: el campo de correo estaba puesto como `enabled = false` (solo lectura), así que la pantalla no permitía «editar los datos del perfil» como pide la guía; y `UpdateProfileRequest` ni siquiera incluía el campo `Email`, de modo que aunque el usuario lo escribiera, nunca se enviaba.

3. Solución aplicada

3.1 Las clases envoltorio

ANTES — NO EXISTÍAN

```
// models/Locations.kt terminaba en la
// data class City. No había ninguna
// clase que representara la envoltura
// que devuelve el servidor.
```

DESPUÉS

```
data class CountriesResponse(
    @SerializedName("countries")
    val countries: List<Country> = emptyList(),
    @SerializedName("count")
    val count: Int = 0
)
```

Se crearon seis clases con este patrón: `CountriesResponse`, `StatesResponse`, `CitiesResponse` (Reto 1), `RolesResponse` y `UserProfileResponse` (Reto 2). **Por qué resuelve el problema:** el tipo ahora describe la estructura real del JSON, así que Gson encuentra el objeto que espera y puede mapearlo.

3.2 El contrato de Retrofit

ANTES

```
@GET("api/locations/countries")
suspend fun getCountries():
    Response<List<Country>>

@GET("api/auth/roles")
suspend fun getRoles():
    Response<List<Role>>
```

DESPUÉS

```
@GET("api/locations/countries")
suspend fun getCountries():
    Response<CountriesResponse>

@GET("api/auth/roles")
suspend fun getRoles():
    Response<RolesResponse>
```

3.3 La lectura en el ViewModel

ANTES

```
countries = response.body()!!

// y el error genérico:
errorMessage = "Error al cargar países"
```

DESPUÉS

```
countries = response.body()!!.countries
Log.d("LOCATION_DEBUG",
    "Países cargados: ${countries.size}")

// el código HTTP real, para distinguir
// un 404 de un 500:
errorMessage = "Error al cargar países " +
    "(código ${response.code()})"
```

3.4 El encadenamiento país → estado → ciudad

La guía pide que al seleccionar un país se disparen sus estados y que se limpie la selección anterior. Antes el `setter` sólo guardaba el país:

ANTES

```
fun updateSelectedCountryForState(
    country: Country?) {
    selectedCountryForState = country
}
```

DESPUÉS

```
fun updateSelectedCountryForState(
    country: Country?) {
    selectedCountryForState = country
    clearMessages()
    // limpia lo de abajo para no dejar
    // datos inconsistentes
    selectedStateForCity = null
    states = emptyList()
    cities = emptyList()
    if (country != null) {
        loadStates(country.id_Country)
    }
}
```

3.5 Validación del formulario de perfil

La validación se puso en el ViewModel y no en la pantalla, para que la regla sea una sola y la pantalla siga limitándose a dibujar el estado que recibe:

```
fun validarFormulario(): String? {
    if (userName.isBlank()) return "El nombre de usuario es requerido"
    if (userName.trim().length < 3) return "El nombre de usuario debe tener al menos 3 caracteres"
    if (email.isBlank()) return "El correo es requerido"
    if (!Patterns.EMAIL_ADDRESS.matcher(email.trim()).matches())
        return "El correo no tiene un formato válido"
    if (selectedCityId <= 0) return "Debes seleccionar una ciudad"
    return null
}
```

El botón «Actualizar Perfil» se apaga mientras `validarFormulario()` devuelva algo distinto de `null`, y el campo de correo muestra el error mientras se escribe.

3.6 Seguridad en dos capas para el cambio de rol

La guía pide comprobar que sólo un Administrador pueda cambiar roles. Se implementó en dos niveles, que es como se hace de verdad:

- **Capa 1, la app.** `UserManagementViewModel` lee `isAdmin` del `SessionManager`. Si el usuario no es administrador, el desplegable de roles queda deshabilitado, aparece un aviso en la pantalla y `updateUserRole()` ni siquiera lanza la petición. Esto es **comodidad**, no seguridad: evita ofrecer una acción que va a fallar.
- **Capa 2, el backend.** `user_routes.py` responde **403** a cualquier petición cuyo token no tenga el rol Administrador. Ésta sí es la seguridad real, porque se cumple aunque alguien salte la interfaz y llame al endpoint directamente con curl o Postman.

La app ahora traduce ese 403 a un mensaje entendible en lugar del genérico anterior:

```
errorMessage = when (response.code()) {
    403 -> "No tienes permisos de administrador para cambiar roles"
    401 -> "Tu sesión expiró, vuelve a iniciar sesión"
    else -> "Error al actualizar el rol (código ${response.code()})"
}
```

4. Un error que cometí y cómo lo encontré

Al cambiar `getCities()` de `Response<List<City>>` a `Response<CitiesResponse>`, **el proyecto dejó de compilar**. La razón es que `ProfileViewModel` también llama a ese mismo método, y seguía haciendo `cities = response.body()!!`, es decir, asignando un objeto `CitiesResponse` a una variable declarada como `List<City>`.

Lo encontré buscando **todos** los llamadores del método antes de dar por terminado el cambio (`Ctrl+Shift+F` sobre `getCities()`). La lección es concreta: cuando se cambia la firma de un método, hay que revisar cada sitio que lo usa, no sólo aquel donde se detectó el problema. En el Reto 2 apliqué esa comprobación desde el principio, y por eso el cambio de `UpdateProfileRequest` —que pasó a exigir un campo nuevo— no rompió nada.

5. Evidencia de verificación

QUÉ SE VERIFICÓ	RESULTADO	DÓNDE ESTÁ LA EVIDENCIA
Los 10 endpoints responden con datos	200 OK	PM1-EV02-captura-endpoints.png
Login correcto devuelve el JWT	200 + token	PM1-EV02-pruebas-endpoints.txt , prueba 5
Login con contraseña errónea	401	prueba 6
Perfil sin token	401	prueba 7
Perfil con token	200 + {"user":...}	prueba 8
Listar usuarios siendo Administrador	200	prueba 9
Listar usuarios sin ser Administrador	403	prueba 10
Editar perfil (PUT)	200 + mensaje	prueba 11
Cambiar rol siendo Administrador	200, rol pasó a Vendedor	prueba 12
Cambiar rol sin ser Administrador	403	prueba 13
Registro de peticiones del servidor	16 peticiones	PM1-EV04-captura-terminal-backend.png

6. Repositorio y ramas

<https://github.com/daniel-roman345/pm1-aplicaciones-mob>

RAMA	COMMITTS	CONTENIDO
main	1	El commit 13f2937 es el proyecto base recibido, sin modificar: sirve para mostrar el estado «antes» durante la sustentación.

RAMA	COMMITTS	CONTENIDO
fix/desplegable-ubicaciones	5	Reto 1. Un commit por paso del razonamiento: primero las clases envoltorio, luego los tipos de retorno, luego la lectura en el ViewModel, luego el encadenamiento y por último el ajuste de ProfileViewModel.
fix/roles-y-perfil	8	Reto 2. Envoltorios de perfil y roles, tipos de retorno, lectura en los dos ViewModel, validación del formulario, correo editable y las dos capas de permisos.

7. Guion de la sustentación (10 minutos)

TIEMPO	QUÉ MOSTRAR Y QUÉ DECIR
0:00 – 1:30	El síntoma. Abrir la pantalla Gestión de Ubicaciones con el código base y mostrar el desplegable vacío. Decir: «un desplegable vacío no es el error, es el síntoma final de una cadena que se rompió en algún punto».
1:30 – 3:00	El recorrido del dato. Recorrerlo en voz alta: base de datos → Flask → Retrofit → Gson → ViewModel → Compose. Señalar que la pregunta del diagnóstico es en qué eslabón se corta.
3:00 – 4:30	El corte. Mostrar <code>curl</code> devolviendo los 5 países con 200 OK. Conclusión: el servidor funciona, luego el problema está del lado de Android. Aquí caen cuatro de las cinco hipótesis.
4:30 – 6:00	La causa raíz. Poner lado a lado el JSON ({ }) y la firma <code>Response<List<Country>></code> ([]). Nombrar la excepción exacta: <code>Expected BEGIN_ARRAY but was BEGIN_OBJECT</code> . Mostrar que usuarios y productos ya usaban envoltorio: era un descuido, no un cambio de la API.
6:00 – 7:30	La corrección en vivo. Mostrar los commits en orden y la app cargando países, seleccionando uno y llenándose el segundo desplegable.
7:30 – 9:00	Reto 2. Editar el perfil (incluido el correo, con la validación funcionando) y cambiar el rol de un usuario. Después entrar con <code>juan_perez</code> y mostrar que el desplegable está deshabilitado y que el backend responde 403. Explicar las dos capas.
9:00 – 10:00	Cierre. «El código compilaba y no funcionaba porque el compilador verifica el contrato que escribió el programador, no el que cumple el servidor. La forma del JSON sólo se conoce en tiempo de ejecución; por eso un error de integración no lo detecta el compilador y hay que mirarlo con Logcat y un cliente HTTP al mismo tiempo.»

8. Preguntas que probablemente hará el instructor

¿Por qué el compilador no detectó el error? Porque `Response<List<Country>>` es Kotlin perfectamente válido. El compilador verifica que el código sea coherente consigo mismo, no que coincida con lo que un servidor externo devuelve. La forma del JSON sólo se conoce cuando la app corre.

¿Qué es un «estado» en Jetpack Compose? Es un valor que, cuando cambia, hace que Compose vuelva a dibujar las funciones que lo leen (recomposición). Se declara con `mutableStateOf`. Por eso una pantalla puede quedarse mostrando una lista vacía aunque los datos

hayan llegado: si el dato se guardó en una variable normal, o en un estado distinto del que la pantalla observa, Compose no se entera de que algo cambió y no vuelve a dibujar.

¿Por qué el mensaje decía «Error de conexión» si la conexión funcionaba? Porque el `catch` (e: `Exception`) del `ViewModel` atrapaba cualquier excepción y le ponía siempre el mismo texto. La excepción real era de `Gson`, no de red. Por eso en la corrección se agregó el `Log.e` con la excepción completa y el código HTTP real en el mensaje: un `catch` genérico esconde la causa.

¿Por qué validar en el `ViewModel` y no en la pantalla? Para que la regla exista una sola vez. Si mañana el perfil se edita desde otra pantalla, la validación viaja con el `ViewModel`. Además mantiene el patrón del proyecto: la pantalla sólo dibuja el estado que recibe.

Si la app ya bloquea el botón, ¿para qué el 403 del backend? Porque la app es código que corre en el teléfono del usuario y no se puede confiar en ella. Cualquiera puede llamar al endpoint con Postman saltándose la interfaz. El bloqueo en la app es comodidad; la seguridad real está en el servidor.